# CECS 229 – Assignment 4

## Linear Congruence Functions

Given the file, 'pa4.py', complete the code for the following functions:

1. `mod_inv(a, m)` – this function returns the inverse of value a under modulus m.
    a. a – is the value that you want to find the inverse of under modulus m. Reduce the value of a until it is in the range [0, m-1].
    b. m – is the modulus. The value of m should be an integer > 0. If it is not, then raise a ValueError.
    c. The returned value should be an integer in the range [0, m-1] that is the inverse of a under modulus m.
    d. Raise a ValueError exception if the gcd of a and m is not 1 (you may use math.gcd() or reuse your gcd function from Assignment 3 to find this value).
    e. Use the Extended Euclidian Algorithm to find the Bézout coefficient of a to find the inverse (you may reuse your bezout_coeffs function from Assignment 3 to find this value). Reduce the result so that the inverse is in the range [0, m-1].
2. `solve_linear_congruence(a, b, m, low, high)` – this function calculates and returns a list of solutions to a linear congruence of the form $ax \equiv b(mod\ m)$.
    a. a – is an integer coefficient of variable x.
    b. b – is an integer congruent to a * x under modulo m.
    c. m – is an integer modulus. The modulus must be > 0.
    d. low – is an integer lower bound for the solution range (inclusive).
    e. high – is an integer upper bound for the solution range (inclusive).
    f. The returned value is a list of all solutions for x in the range [low, high].
    g. Raise a ValueError if high < low.
    h. Loop through and test each of the values in the range to see if it is a solution for the linear congruence. If it is, add it to the list.

**Example Input/Output**:

```
mod_inv(3, 11)
4
mod_inv(9, 19)
17
solve_linear_congr(3, 4, 7, -5, 5)
[-1]
solve_linear_congr(3, -7, 29, -50, -3)
[-41, -12]
```

**Notes:**

1. Please place your name in the comment block at the top of the provided file.
2. Fill in the necessary code to complete the functions. Write your code where specified (ie. the TODOs). Do not modify the file name, function names, or parameters otherwise the program and tests will not work. Return the results in the format specified (ie. correct types and container types), otherwise the program and tests will not work properly.

3. Your solutions may not use any libraries or functions that perform the equivalent task. You may use math.gcd() for mod_inv(), and/or you can reuse your gcd and bezout_coeff() function that you wrote for Assignment 3.
4. Test your code using the provided 'main.py' file. You can write additional tests in the 'your_tester.py' file to further check your solutions. Note: just because the main tests pass does not guarantee that it will pass the tests on Gradescope.
5. When you are finished, submit your 'pa4.py' file to Gradescope on Canvas for further testing and grading. It should allow you to make repeated submissions.